Conversational AI Chatbot using Natural Language Processing (NLP)

Building Intelligent Human-Computer Conversations

INTERNSHIP PROJECT

Presented By:

Tanushree Parakh

B.Tech – Electrical & Electronics Engineering

Developed using:

PYTHON | NLP | Scikit-Learn | STREAMLIT

Introduction to NLP Chatbots

1

What are Chatbots?

Automated conversational agents designed to simulate human interaction in real time.

2

What is NLP?

Natural Language Processing enables computers to understand, interpret, and generate human language.

3

Intelligent Chatbots

Use NLP techniques such as tokenization, stopword removal, lemmatization, and similarity matching to understand user queries and provide relevant responses.

4

Applications

Widely used in education, customer support, virtual assistants, and information retrieval systems to provide instant and accurate responses.

5

Project focus

This project implements a Conversational AI Chatbot using Python, NLTK, TF-IDF Vectorization, and Cosine Similarity to answer user queries in natural language.

Project Objectives

The project aims to develop a Conversational AI Chatbot that can understand natural language queries and generate relevant responses using NLP techniques such as tokenization, lemmatization, TF-IDF vectorization, and cosine similarity.

Natural Language Understanding

Develop a chatbot capable of understanding user queries in natural language using NLP techniques such as tokenization, stopword removal, and lemmatization.

Accurate Response Generation

Provide relevant and meaningful responses by identifying the most similar question in the knowledge base and returning the corresponding answer.

Intelligent Information Retrieval

Utilize TF-IDF vectorization and cosine similarity to identify and retrieve the best matching response efficiently.

Interactive User Experience

Create a simple and user-friendly chatbot interface using Streamlit for seamless interaction.

Technologies Used

Python

Core programming language used for developing the chatbot and implementing NLP functionality.

NLTK

Used for text preprocessing tasks such as stopword removal and lemmatization.

Scikit-Learn

Used for TF-IDF Vectorization and Cosine Similarity to identify the most relevant response.

Pandas

Used to load, read, and manage the chatbot knowledge base stored in CSV format.

Streamlit

Used to create an interactive and user-friendly web interface for the chatbot.

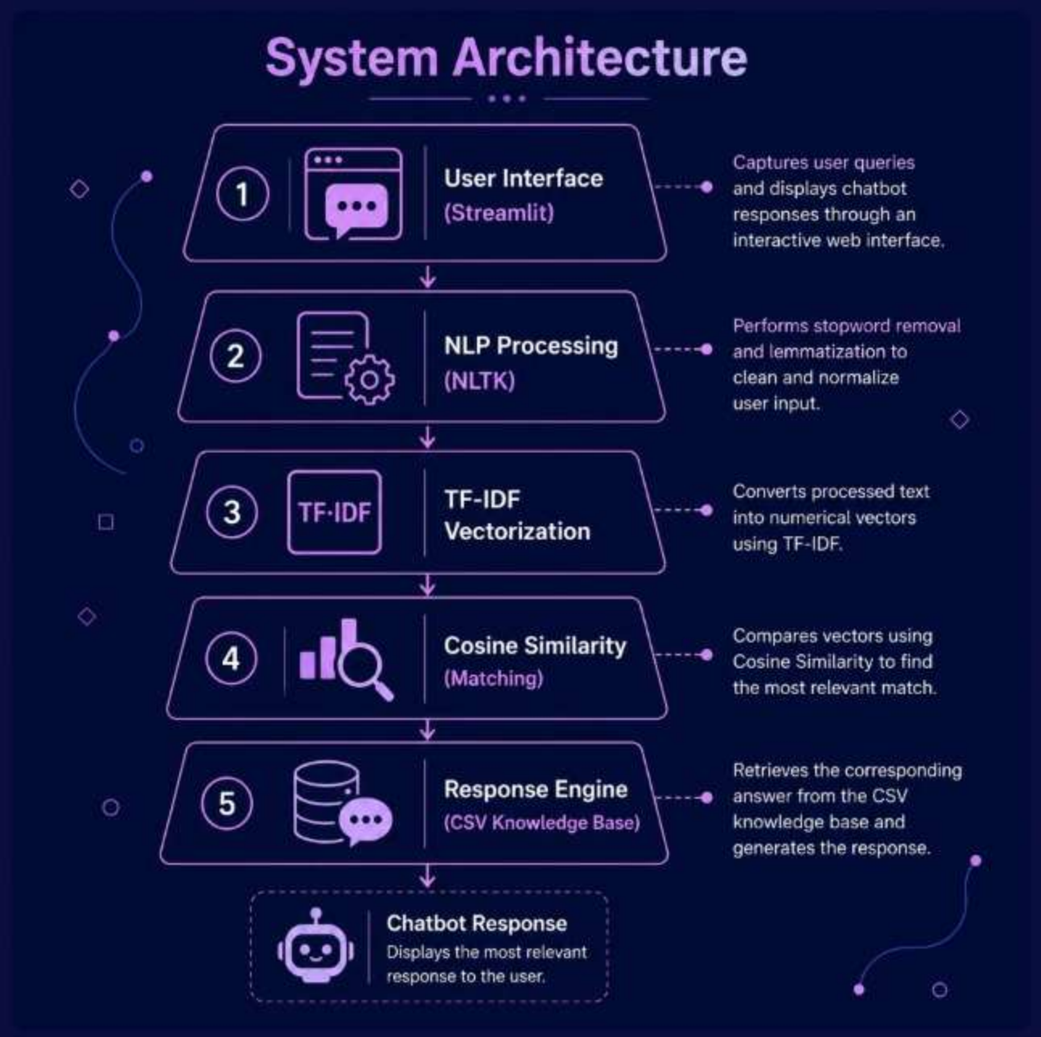
CSV Knowledge Base

Stores predefined questions and answers that are used to generate chatbot responses.

System Architecture

Architecture Overview

The chatbot follows an NLP-based workflow that processes user queries, retrieves the most relevant information from the knowledge base, and generates meaningful responses.



UI (Streamlit)

Captures user queries and displays chatbot responses through an interactive web interface.

NLP Engine

Performs stopword removal and lemmatization to clean and normalize user input.

TF-IDF Engine

Uses TF-IDF Vectorization and Cosine Similarity to identify the best matching question.ed intents and responses.

Response Engine

Retrieves the corresponding answer from the CSV knowledge base and generates the chatbot response..

Chatbot Workflow

Every interaction follows a structured NLP workflow that transforms user input into a relevant response. Understanding this process helps explain how the chatbot analyzes queries and retrieves the most appropriate answer.

This seven-step workflow enables the chatbot to process natural language queries, identify the most relevant information, and provide accurate responses in real time.

Implementation Snapshot: Building the Chatbot

The chatbot is implemented using Python with NLP libraries and a lightweight web interface for real-time interaction.

1

Environment Setup

Install Python 3.8+ and required libraries using pip:

```
pip install nltk streamlit scikit-learn
```

2

NLP Data Processing Setup

- Use NLTK for text preprocessing
- Tokenization, stopword removal, and normalization
- Convert text into structured format for model training

3

Core Logic Development

- Input preprocessing and normalization
- Intent classification using Scikit-learn
- Feature extraction using TF-IDF
- Response generation using predefined templates

4

UI Development

- Built interactive chat interface using Streamlit
- Enables real-time user input and bot responses
- Lightweight web app for quick testing and deployment

Code Spotlight – NLP Implementation

First, the user query is preprocessed using stopword removal and lemmatization. Then TF-IDF converts text into vectors. Finally, cosine similarity finds the most relevant question and returns the corresponding answer.

1

Text Preprocessing

```
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

lemmatizer = WordNetLemmatizer()
stop_words =
set(stopwords.words('english'))

def preprocess(text):
    words = text.lower().split()

    words = [
        lemmatizer.lemmatize(word)
        for word in words
        if word not in stop_words
    ]

    return " ".join(words)
```

2

TF-IDF Vectorization

```
from sklearn.feature_extraction .text import
TfidfVectorizer

vectorizer = TfidfVectorizer()

question_vectors = vectorizer.fit_transform(
    processed_questions
)
```

Converts textual questions into numerical vectors using TF-IDF Vectorization.

3

Cosine Similarity Matching

```
from sklearn.metrics.pairwise import
cosine_similarity

user_vector =
vectorizer.transform([processed_input])

similarity = cosine_similarity(
    user_vector,
    question_vectors
)

best_match = similarity.argmax()
```

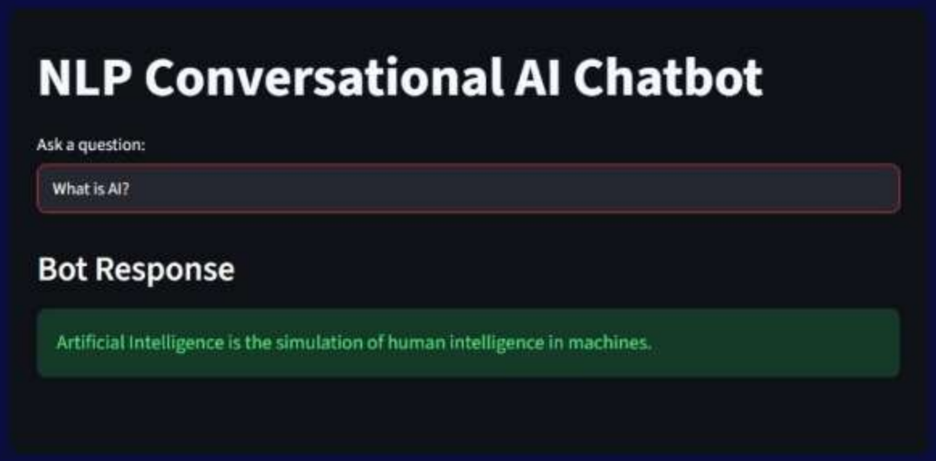
Finds the most relevant question from the knowledge base using Cosine Similarity.

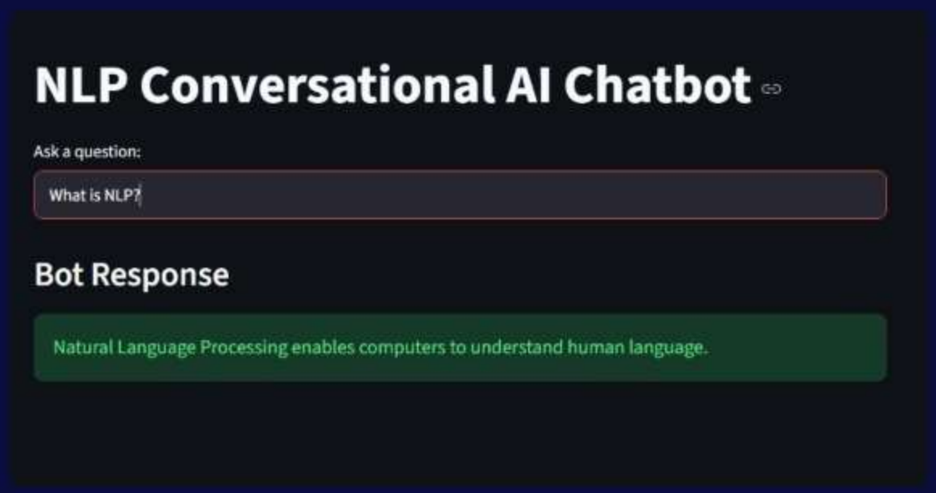
NLP Techniques Implemented:

- Stopword Removal
- Lemmatization
- TF-IDF Vectorization
- Cosine Similarity Matching

Results & Future Scope

RESULTS:





- The chatbot successfully understands natural language queries and provides accurate, real-time responses using NLP techniques. It performs well in identifying user intent and delivering relevant answers, demonstrating effective and smooth human-like interaction.

Future Scope:

- **Voice Support**

  Enable speech-to-text and text-to-speech for hands-free chatbot interaction

- **Multi-language Support**

  Allow users to interact with the chatbot in multiple regional and global languages

- **Cloud Deployment**

  Deploy the chatbot on cloud platforms for scalability, availability, and real-time access

- **Generative AI Integration**

  Integrate advanced Generative AI models to improve response quality and contextual understanding

Conclusion & Thank You

This project successfully demonstrates the development of a Conversational AI Chatbot using Natural Language Processing techniques. The chatbot is capable of understanding user queries and retrieving relevant responses using TF-IDF Vectorization and Cosine Similarity matching. It highlights how NLP can be effectively used to bridge the gap between human language and machine understanding, enabling faster and more intuitive information access.

- **Key Takeaway**

  NLP-based chatbots significantly improve human-computer interaction by enabling efficient, scalable, and automated response systems.

- **Technology Stack**

  Python, NLTK, Scikit-Learn, and Streamlit together provide a simple and efficient framework for building NLP-based chatbot applications.

- **Next Steps**

  Future improvements include voice-based interaction, multi-language support, and cloud deployment for better scalability and accessibility.

Thank you for your time and attention.